

EXP 6 : Feedforward and Convolutional Neural Networks

Aim

To demonstrate the construction and application of a **Feedforward Neural Network (FNN)** for classification and a **Convolutional Neural Network (CNN)** for image classification using Keras with TensorFlow backend.

Algorithm

FNN

1. Load the Fashion-MNIST dataset.
2. Flatten the images.
3. Normalize pixel values.
4. Build the FNN using dense layers.
5. Compile the model using Adam optimizer.
6. Train the model.
7. Evaluate the model using accuracy.
8. Generate classification report and confusion matrix.

CNN

1. Load the MNIST dataset.
2. Reshape and normalize the images.
3. Apply convolution and ReLU activation.
4. Apply max pooling.
5. Flatten the feature maps.
6. Add fully connected layers.
7. Train and evaluate the CNN.
8. Generate classification report, confusion matrix and predictions.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

```

from tensorflow.keras.datasets import mnist, fashion_mnist
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
tf.keras.utils.disable_interactive_logging()
print("TensorFlow Version:", tf.__version__)
print("\n--- FNN ---")
(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) = fashion_mnist.load_data()
print("Training data:", x_train_fnn.shape)
print("Testing data:", x_test_fnn.shape)
x_train_fnn = x_train_fnn.reshape(-1, 28 * 28)
x_test_fnn = x_test_fnn.reshape(-1, 28 * 28)
x_train_fnn = x_train_fnn / 255.0
x_test_fnn = x_test_fnn / 255.0
model_fnn = keras.Sequential([
    layers.Dense(128, activation="relu", input_shape=(784,)),
    layers.Dropout(0.2),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
model_fnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model_fnn.summary()
history_fnn = model_fnn.fit(
    x_train_fnn,
    y_train_fnn,
    epochs=10,
    validation_split=0.1,
    verbose=1
)
loss_fnn, accuracy_fnn = model_fnn.evaluate(
    x_test_fnn,
    y_test_fnn,
    verbose=0
)
print("\nFNN Test Loss:", round(loss_fnn, 4))
print("FNN Test Accuracy:", round(accuracy_fnn, 4))
y_pred_fnn = np.argmax(

```

```

    model_fnn.predict(x_test_fnn, verbose=0),
    axis=1
)
print("\nFNN Classification Report")
print(classification_report(y_test_fnn, y_pred_fnn))
cm_fnn = confusion_matrix(
    y_test_fnn,
    y_pred_fnn
)
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm_fnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
)
plt.title("FNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

plt.figure(figsize=(7, 5))
plt.plot(
    history_fnn.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history_fnn.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.title("FNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(7, 5))
plt.plot(
    history_fnn.history["loss"],

```

```

        label="Training Loss"
    )
    plt.plot(
        history_fnn.history["val_loss"],
        label="Validation Loss"
    )
    plt.title("FNN Model Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)
    plt.show()

```

```

print("\n--- CNN ---")
(x_train_cnn, y_train_cnn), (x_test_cnn, y_test_cnn) = mnist.load_data()

```

```

print("Training data:", x_train_cnn.shape)
print("Testing data:", x_test_cnn.shape)
x_train_cnn = x_train_cnn.reshape(
    x_train_cnn.shape[0],
    28,
    28,
    1
)
x_test_cnn = x_test_cnn.reshape(
    x_test_cnn.shape[0],
    28,
    28,
    1
)
x_train_cnn = x_train_cnn.astype("float32") / 255.0
x_test_cnn = x_test_cnn.astype("float32") / 255.0

```

```

model_cnn = keras.Sequential([
    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    )
])

```

```

),
layers.MaxPooling2D((2, 2)),
layers.Conv2D(
    64,
    (3, 3),
    activation="relu"
),
layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(
    128,
    activation="relu"
),
layers.Dropout(0.5),
layers.Dense(
    10,
    activation="softmax"
)
])
model_cnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model_cnn.summary()
history_cnn = model_cnn.fit(
    x_train_cnn,
    y_train_cnn,
    epochs=10,
    validation_split=0.1,
    verbose=1
)
loss_cnn, accuracy_cnn = model_cnn.evaluate(
    x_test_cnn,
    y_test_cnn,
    verbose=0
)
print("\nCNN Test Loss:", round(loss_cnn, 4))
print("CNN Test Accuracy:", round(accuracy_cnn, 4))
y_pred_cnn = np.argmax(

```

```

    model_cnn.predict(x_test_cnn, verbose=0),
    axis=1
)

print("\nCNN Classification Report")
print(classification_report(y_test_cnn, y_pred_cnn))
cm_cnn = confusion_matrix(
    y_test_cnn,
    y_pred_cnn
)
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm_cnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
)
plt.title("CNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
plt.figure(figsize=(7, 5))
plt.plot(
    history_cnn.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history_cnn.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.title("CNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(7, 5))
plt.plot(
    history_cnn.history["loss"],

```

```

        label="Training Loss"
    )
    plt.plot(
        history_cnn.history["val_loss"],
        label="Validation Loss"
    )
    plt.title("CNN Model Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)
    plt.show()
    print("\n--- Sample CNN Predictions ---")
    plt.figure(figsize=(10, 10))
    for i in range(25):
        plt.subplot(5, 5, i + 1)
        plt.imshow(
            x_test_cnn[i].reshape(28, 28),
            cmap="gray"
        )
        plt.title(
            f"True: {y_test_cnn[i]}\nPred: {y_pred_cnn[i]}"
        )
        plt.xticks([])
        plt.yticks([])

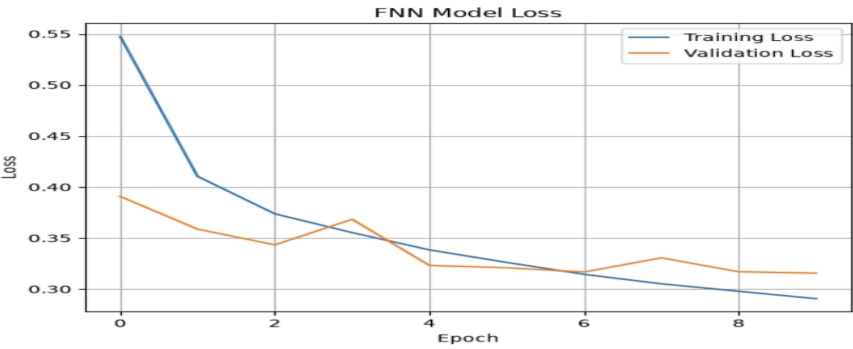
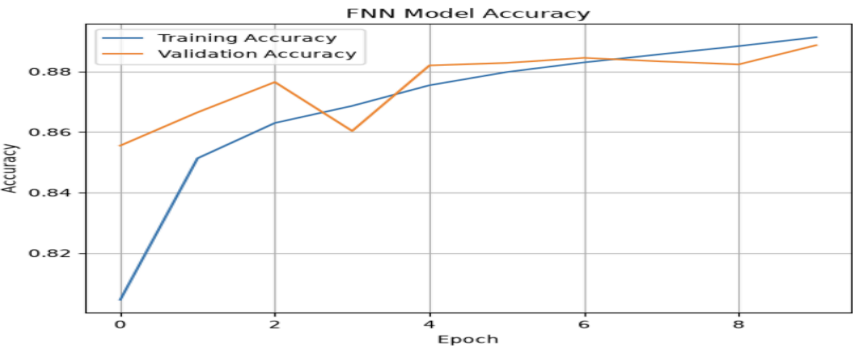
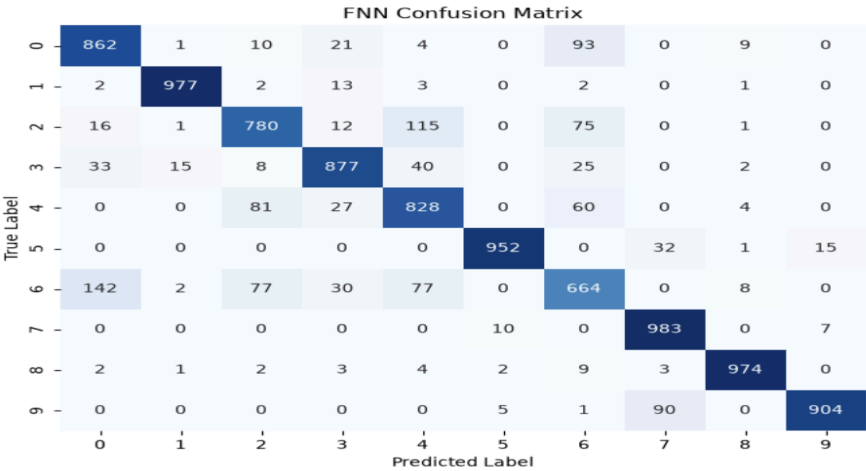
    plt.tight_layout()
    plt.show()
    print(
        "FNN Accuracy:",
        round(accuracy_fnn * 100, 2),
        "%"
    )
    print(
        "CNN Accuracy:",
        round(accuracy_cnn * 100, 2),
        "%"
    )

```

Output:

FNN Test Loss: 0.343
FNN Test Accuracy: 0.8801

FNN Classification Report				
	precision	recall	f1-score	support
0	0.82	0.86	0.84	1000
1	0.98	0.98	0.98	1000
2	0.81	0.78	0.80	1000
3	0.89	0.88	0.88	1000
4	0.77	0.83	0.80	1000
5	0.98	0.95	0.97	1000
6	0.71	0.66	0.69	1000
7	0.89	0.98	0.93	1000
8	0.97	0.97	0.97	1000
9	0.98	0.90	0.94	1000
accuracy			0.88	10000
macro avg	0.88	0.88	0.88	10000
weighted avg	0.88	0.88	0.88	10000

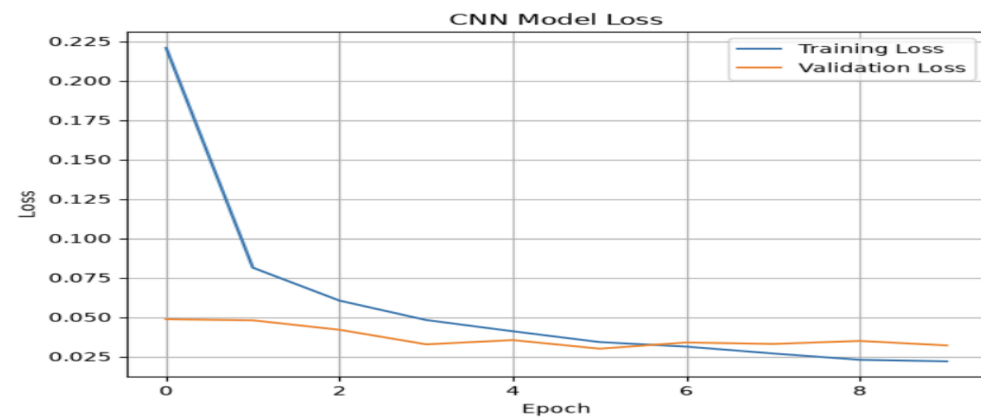
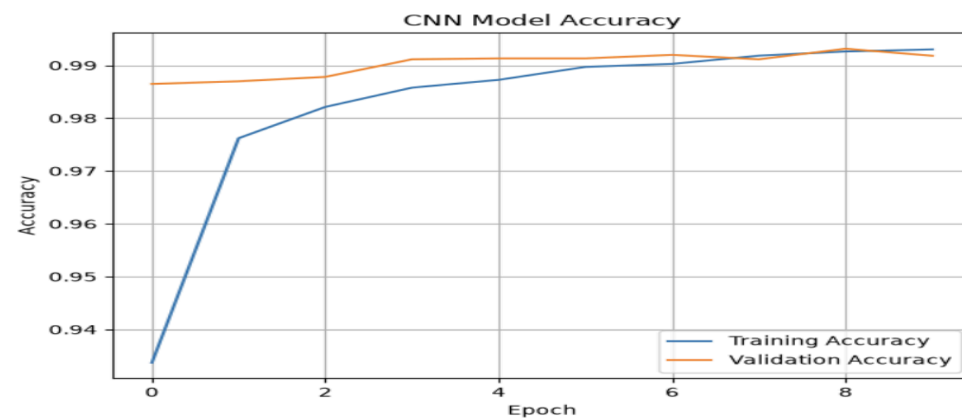


---- CNN ----
Training data: (60000, 28, 28)
Testing data: (10000, 28, 28)

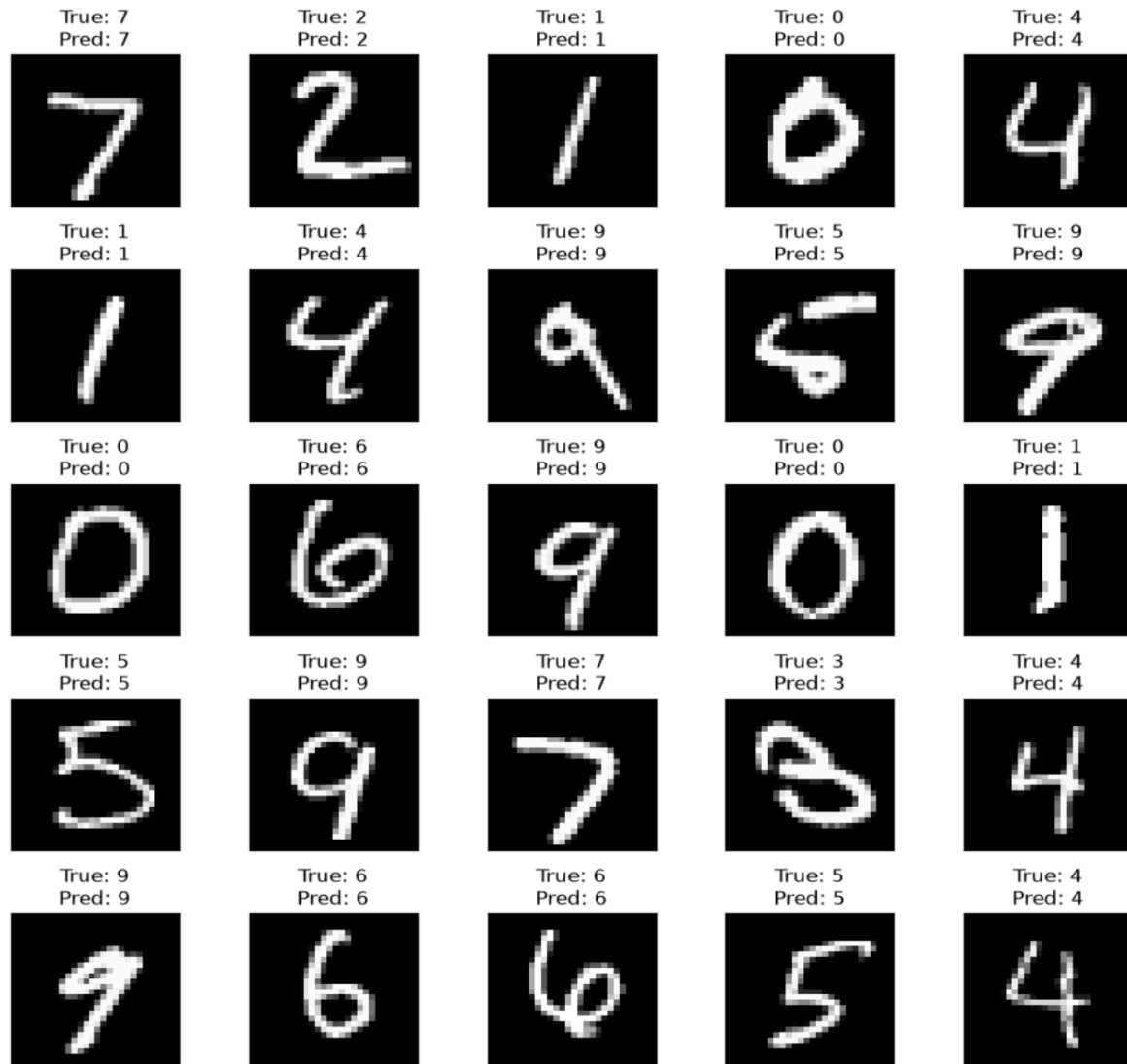
CNN Test Loss: 0.026
 CNN Test Accuracy: 0.9933

	precision	recall	f1-score	support
0	1.00	1.00	1.00	980
1	0.99	1.00	1.00	1135
2	0.99	1.00	0.99	1032
3	1.00	0.99	0.99	1010
4	1.00	0.99	1.00	982
5	0.98	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

	0	1	2	3	4	5	6	7	8	9
0	977	0	0	0	0	0	1	1	1	0
1	0	1134	1	0	0	0	0	0	0	0
2	0	1	1031	0	0	0	0	0	0	0
3	0	0	2	1003	0	4	0	0	1	0
4	0	0	0	0	977	0	1	0	0	4
5	1	0	0	4	0	885	1	1	0	0
6	2	3	0	0	1	6	946	0	0	0
7	0	3	5	0	0	0	0	1019	0	1
8	1	0	2	1	0	1	0	2	965	2
9	0	0	0	0	2	5	1	2	3	996
	0	1	2	3	4	5	6	7	8	9



--- Sample CNN Predictions ---



FNN Accuracy: 88.01 %
CNN Accuracy: 99.33 %

Result:

The FNN and CNN models were successfully implemented and evaluated. The FNN learned classification using fully connected layers, while the CNN extracted spatial features using convolution and pooling layers. The CNN is designed for image classification and demonstrated the ability to learn spatial patterns from image data.